

Resumen

Revisión exhaustiva de 8 actividades de Flexbox, detallando correcciones cruciales, seguida de una introducción a CSS Grid para el diseño bidimensional.

Revisión de actividades Flexbox

Se revisaron varias actividades de Flexbox, confirmando la correcta implementación de centrado (`align-items`, `justify-content`) e inversión de dirección. La Actividad 4 requirió la adición de `flex-grow` en el párrafo y ajustes de imagen para corregir desbordes.

Introducción a CSS Grid

Se introdujo CSS Grid, destacando su soporte bidimensional (filas y columnas) a diferencia de Flexbox. Se explicaron conceptos clave como las unidades de fracción (Fr), grillas explícitas vs. implícitas, y la función `repeat`.

Combinación Grid y Flexbox

Se demostró la implementación de diseños complejos con Grid, incluyendo el uso de áreas nombradas y el manejo de contenido dinámico. Se enfatizó que Grid es ideal para estructuras macro y Flexbox para micro-diseños internos.

- **Revisión de la Actividad 1 (Flexbox):** Santiago Verrier propuso que cada estudiante proyectara la resolución de las actividades para su revisión colectiva antes de comenzar con Grid. Aroa Ramirez López se ofreció a mostrar la Actividad 1, la cual fue considerada la más fácil. Se confirmó que la solución era correcta al ver las propiedades y valores utilizados, especialmente para centrar elementos vertical y horizontalmente (usando ``align-items`` y ``justify-content``) e invertir la dirección (``flex-direction``).
- **Interrupción y Continuación de la Revisión de Actividades:** Santiago Verrier se disculpó por una breve interrupción debido a una llamada del jardín de su hijo. Tras la interrupción, se reorganizó el orden de presentación de actividades: Alex Maxenchs mostraría la Actividad 2, y luego Alex González

Arias De Saavedra presentaría la Actividad 5. Sin embargo, luego se estableció un orden diferente: Alex, Claudia, Jime y después Alex y Sebas.

- **Revisión de la Actividad 4 (Tarjetas):** Jimena Espejo compartió su pantalla para revisar la Actividad 4, que trataba sobre tarjetas con contenido variable y un botón fijo en la parte inferior. Se identificó que faltaba la propiedad ``flex-grow`` en el párrafo (como hijo) para permitir que creciera y empujara el botón hacia abajo. También se recomendó remover ``align-items: center`` del contenedor para evitar centrar los tres elementos hijos y usar ``align-self`` en el botón "Leer más" para centrarlo.
- **Correcciones en la Actividad 4 (Imágenes):** Se abordó el problema de la imagen que se salía de su contenedor padre. La solución presentada fue aplicar ``width: 100%`` y ``height: auto`` a la imagen para que mantuviera su tamaño dentro del ancho de su padre. Jimena Espejo procedió a aplicar un ``reset`` de asterisco (``*``) para eliminar espacios predeterminados y manejar el diseño de manera más controlada.
- **Revisión de la Actividad 5 (Hero):** Sebastian J. Alonso presentó la Actividad 5, el componente hero, que quedó centrado en la vista. El código fue revisado y aprobado por Santiago Verrier, quien señaló que el uso de ``width: 100%`` en el `*body*` era redundante, pero que el uso de ``display: flex``, ``flex-direction: column``, ``justify-content`` y ``align-items`` para centrar estaba perfecto.
- **Revisión de la Actividad 6 (Menú de Comidas):** Alex Gonzalez Arias De Saavedra mostró la Actividad 6, el menú de comidas, elogiando la implementación visual y el efecto `*hover*`. Se realizó una corrección al uso de ``transition`` en el CSS, cambiando ``color`` a ``all`` para que la transición aplicara también al cambio de ``font-size`` y a todas las demás propiedades modificadas. Alex Gonzalez Arias De Saavedra también mencionó la dificultad de usar párrafos en lugar de ``span`` para alinear los precios, ya que el párrafo ocupa toda la línea.
- **Técnicas de Separación de Elementos en el Menú:** Santiago Verrier recordó la utilidad del truco de ``margin-left: auto`` o ``margin-right: auto`` en elementos flex para separarlos automáticamente a los lados opuestos, similar a la navegación. La solución de Alex Gonzalez Arias De Saavedra con ``span`` o un ``div`` también era considerada correcta.
- **Revisión de la Actividad 2 (Barra de Navegación):** Claudia Val Lecha y Alex Maxenchs mostraron la Actividad 2, la barra de navegación con un `*login*`. La solución, que incluía ``display: flex``, ``align-items: center`` (para centrar verticalmente), y ``justify-content: space-between`` (para separar horizontalmente), fue considerada muy bien lograda.

- **Revisión de la Actividad 3 (Galería):** Sebastian J. Alonso presentó la Actividad 3, la galería, reconociendo que le había costado y que las proporciones no estaban perfectas, lo cual fue considerado normal. Santiago Verrier sugirió utilizar la función ``calc()`` para el ancho de los elementos, como ``calc(50% - 5px)`` para ajustar por el `*gap*`. Se demostró la complejidad de implementar la galería responsiva solo con Flexbox y la dificultad de manejar los espacios generados por el `*gap*`.
- **Introducción a CSS Grid y su Contexto Histórico:** Después de un breve receso, Santiago Verrier introdujo Grid, situando su aparición en octubre de 2017, aproximadamente dos años después de Flexbox, según caniuse.com. Se destacó que Grid ofrece soporte para dos dimensiones (filas y columnas), a diferencia de Flexbox que se centra en una sola dimensión (horizontal o vertical).
- **Similitudes entre Flexbox y Grid:** Grid es una herramienta moderna diseñada para la responsividad, y al igual que Flexbox, utiliza la propiedad ``display`` (como ``display: grid``) y tiene propiedades para el contenedor y los elementos hijos. Santiago Verrier destacó que muchas propiedades y valores de Grid, como ``justify-items``, ``align-items``, ``align-content`` y ``justify-content``, comparten nombres similares con las de Flexbox, lo cual puede resultar confuso inicialmente debido a la cantidad de opciones.
- **Propiedades de Flexbox y Grid:** Santiago Verrier comenzó la discusión repasando las múltiples posibilidades de las grillas, incluyendo el uso de ``gap``, ``align-self``, y ``justify-self``. Se notó que la complejidad de estas propiedades radica en la dificultad de comprender su aplicación hasta que no se visualizan ejemplos prácticos, ya que cada ítem en la grilla actúa como si estuviera en una celda que virtualmente funciona como un contenedor padre.
- **Introducción a las Fracciones (Fr):** Se introdujo una nueva unidad específica para Grid, denominada "fracciones" (Fr), que permite trabajar con la distribución de espacio. Las fracciones facilitan la repartición del espacio disponible en partes proporcionales dentro de la grilla, lo cual es más sencillo de calcular que los porcentajes, especialmente cuando se buscan divisiones desiguales.
- **Creación de Grillas Explícitas e Implícitas:** Se explicó la diferencia entre las grillas explícitas, que son las especificadas directamente en el código, y las grillas implícitas, que son las filas que Grid crea automáticamente cuando el contenido se desborda. Santiago Verrier señaló que, aunque las columnas pueden definirse explícitamente, las filas implícitas pueden recibir una altura uniforme usando ``grid-auto-rows``, evitando la necesidad de contarlas una por una.

- **Recursos Adicionales y Ejemplos de Uso:** Santiago Verrier mencionó que compartirían recursos como gráficos y una guía ("cheat sheet") de Grid y CSS, destacando que los desarrolladores deben consultar constantemente la documentación. Se presentaron ejemplos prácticos, como la creación de diseños complejos (ej. una cabecera de red social) que serían difíciles de lograr solo con Flexbox.
- **Uso de Herramientas de Inspección:** Se demostró la utilidad del inspector de Firefox para visualizar cómo Grid sombrea las áreas y permite la superposición de celdas y elementos, lo que puede ser práctico para ciertas necesidades de diseño. Santiago Verrier también mostró cómo Chrome visualiza las líneas numeradas de la grilla para ayudar en los cálculos de posicionamiento.
- ****Implementación con ``grid-template-columns`` y ``repeat``**:** Para crear columnas de manera más flexible, se mostró cómo usar ``grid-template-columns`` con píxeles o fracciones. Se destacó la conveniencia de la función ``repeat``, que permite especificar rápidamente un número de columnas idénticas con el valor de una fracción, simplificando la definición del diseño de la grilla.
- ****Uso de Áreas Nombradas (``Grid Template Areas``) y Líneas**:** Se explicó que se pueden asignar nombres a las áreas de la grilla (ej. ``header``, ``sidebar``) para estructurar el diseño, permitiendo que un elemento ocupe varias celdas. También se detalló el uso de líneas numeradas para posicionar elementos, indicando el inicio y el final de un ítem en las filas y columnas, lo que es útil para superponer capas o definir áreas exactas.
- **Demostración de Galería de Imágenes Sencilla:** Santiago Verrier mostró cómo crear una galería de imágenes simple utilizando ``display: grid`` con la función ``repeat`` para definir las columnas. Se destacó que ``gap`` en Grid funciona de manera eficiente para separar los bloques sin problemas de espaciado, a diferencia de Flexbox, lo que resulta en un diseño limpio.
- **Desplazamiento de Elementos al Agrandar una Imagen:** Se demostró que al aumentar explícitamente el tamaño de una imagen usando coordenadas de línea (``grid-row-start`/`end`` y ``grid-column-start`/`end``), los demás ítems se desplazan automáticamente para llenar las celdas restantes. Este comportamiento es útil para galerías donde se desea destacar un elemento, ya que Grid gestiona el desplazamiento del resto del contenido.
- **Estrategia de Uso Combinado de Grid y Flexbox:** En respuesta a una pregunta sobre por qué Grid no reemplaza completamente a Flexbox, se explicó que Grid es ideal para el diseño macro (estructurar la página en grandes bloques), mientras que Flexbox se utiliza mejor para el control de detalles internos o

micro-diseños dentro de esos bloques, como la alineación de elementos hijos. Ejemplos de sitios web reales mostraron esta combinación, utilizando Grid para la estructura general y Flexbox para la distribución detallada de contenido dentro de los bloques.

- **Discusión sobre la aplicación de Grid y Gap:** Santiago Verrier y Marcos Oliveros discutieron la utilidad de la implementación de `grid` para el diseño de bloques, señalando que esta técnica permite evitar el uso de `padding` y `margin` mediante la aplicación de `gap`. Santiago Verrier señaló el uso de `span` como una propiedad más avanzada para especificar el espacio que ocupará un elemento en filas y columnas. Marcos Oliveros consideró que esta característica es "una maravilla".
- **Exploración de técnicas de diseño en un ejemplo web:** Santiago Verrier examinó el código de un sitio web para determinar las técnicas de diseño empleadas, inicialmente buscando `flexbox` pero encontrando que el desarrollador usó `inline-block` para los elementos de lista (los `li`) en lugar de `flexbox`. Explicó una interrupción debido a un problema con los timbres del edificio y luego procedió a buscar las propiedades de diseño, notando que la elección de usar una lista para noticias o componentes fue una decisión arbitraria del desarrollador. Santiago Verrier también revisó el uso de clases y notó una convención de nombramiento con mayúsculas en el código, sugiriendo que esto podría ser una práctica específica del equipo de desarrollo.
- **Análisis de la estructura de cuadrícula y la combinación de Flexbox y Grid:** Santiago Verrier analizó un ejemplo de cuadrícula (Grid) para personajes de Marvel, observando que la estructura general usaba una cuadrícula simple y que el sitio de 81 páginas probablemente utilizaba contenido dinámico. Señaló que el diseño combinaba `Flexbox` en la parte inferior para elementos específicos y `Grid` para los logotipos, mostrando cómo `space between` se utiliza para la separación y que las navegaciones internas a menudo usan listas normales sin necesidad de `Flexbox` o `Grid`. Santiago Verrier sugirió visitar sitios web e inspeccionar su código para entender mejor cuándo usar `Flexbox` y cuándo usar `Grid`.
- **Demostración del uso de áreas en Grid Layout:** Santiago Verrier mostró un ejemplo de diseño donde se utilizaron `grid template areas` en lugar de líneas para definir la estructura de la cuadrícula, señalando que esta es una decisión del desarrollador. El ejemplo demostró cómo se nombran las áreas (`logo`, `picture`, `tagline`) para construir el esquema de la cuadrícula y cómo esto puede definir el diseño visual de la página. Luego, mostró la construcción de un ejemplo práctico en Visual Studio Code para recrear un diseño típico con

encabezado, menú lateral (`aside`), contenido principal (`main`) y pie de página, utilizando áreas y especificando el tamaño de las columnas.

- **Implementación de diseño usando áreas y manejo de contenido dinámico:**
Santiago Verrier procedió a construir el esquema de áreas en el código, asignando nombres como `encabezado`, `prince`, `publ` (publicidad) y `pie` a las celdas. Al establecer las filas con `auto`, explicó que esto permite que el contenido calcule automáticamente el tamaño de la fila. Alex Gonzalez Arias De Saavedra preguntó cómo manejar el espacio sobrante si el contenido de la publicidad es más corto que el contenido principal, especialmente con un ajuste automático de filas.

Práctica guiadas cuando tengan tiempo disponible

Seguir éste tutorial (no es algo perfecto pero tienen la chance de mejorarlo y tomar sus decisiones de mejora en tanto CSS y estructura ¿usar flex y/o grid?)

[Tutorial de CSS Flexbox y CSS Grid: Cómo crear una página de inicio responsivo con HTML y CSS](#)

Alternativa [How To Combine Flexbox and CSS Grids for Efficient Layouts](#)

alternativa2 [How to create an image gallery with CSS Grid](#)

Material didáctico

- Grid
 - **Artículos**
 - unidad nueva, llamada ["fr" \(fracción\)](#) -
[Guide to CSS Grid Layout Fr Unit - Hongkiat](#)
 - [Grid by Example](#)
 - [A Complete Guide to Grid | CSS-Tricks](#)
 - [The Difference Between Explicit and Implicit Grids | CSS-Tricks](#)
 - [Explicit vs Implicit Grid](#)
 - [How to Use a Grid in Web Design](#)
 - [Resolviendo problemas con CSS Grid y Flexbox: Diseño de interfaz mediante tarjetas](#)
 - **Juegos**
 - [Grid Garden](#)

- [Grid Critters](#)
- [Play Grid Attack – CSS Game to learn CSS Grid](#)
- **Estructuras**
 - [Create a Website Layout with CSS Grid](#)
 - [CSS Grid Examples](#)
 - [CSS Grid Generator](#)
 - [Grid Layoutit](#)
 - [An Introduction to CSS Grid Layout \(with Examples\)](#)
 - Ejemplos de sitios
 - [Grid masterclass](#)
 - [BBC News Mundo - Noticias](#)
 - [Marvel Characters, Super Heroes, & Villains List](#)
 - [Des Climbing Gym](#)
- **Ejemplos de propiedades**
 - *propiedades padre*
 - display grid
 - [grid-template-columns](#)
 - [grid-template-rows](#)
 - [grid-template-areas](#)
 - [column-gap](#)
 - [row-gap](#)
 - Propiedades hijo
 - [grid-column-start](#)
 - [grid-column-end](#)
 - [grid-row-start](#)
 - [grid-row-end](#)
 - [grid-area](#)
- **Ejemplos prácticos**
 - [Profile Card Layout](#) (ref: [original](#))
 - [Overlap use cases](#) (ref: [Original](#))
 - [Card with text reveal on over](#)
 - [How to create an image gallery with CSS Grid](#)